worked_example

June 10, 2025

1 A Toy CNN

We have devised a CNN architecture similar to the one we will be implementing in hardware. In this version, we have reduced the bitwidth, channels, timesteps and magnitude of convolutions. The intention is to show how all the operations fit together to perform inference in our CNN, but in a digestable format.

1.0.1 Step 1

We generate the weight arrays randomly, sampling from a kaiming_uniform distribution. This is how PyTorch would initialize weights prior to training, but we will consider these the actual float trained weights for our model.

[1]:
```
import numpy as np

np.random.seed(42)

def kaiming_uniform(shape, fan_in):
    bound = np.sqrt(1.0 / fan_in)
    return np.random.uniform(-bound, bound, shape)

# Layer 1: Depthwise Conv1d (2 channels, kernel size 5)
DW1 = kaiming_uniform((2, 5), fan_in=5)  # Each depthwise kernel sees only its␣
↪own channel

# Layer 2: Pointwise Conv1d (4 output channels, 2 input channels)
PW1 = kaiming_uniform((4, 2), fan_in=2)  # Fully connected across input channels␣
↪at each spatial position

# Layer 3: Depthwise Conv1d (4 channels, kernel size 3)
DW2 = kaiming_uniform((4, 3), fan_in=3)  # Again, each kernel sees only its own␣
↪channel

# Layer 4: Pointwise Conv1d (6 output channels, 4 input channels)
PW2 = kaiming_uniform((6, 4), fan_in=4)

# Fully Connected: 2 output, 6 input
FC = kaiming_uniform((2, 6), fan_in=6)
```

```python
# FC Bias: Uniform bias init (PyTorch uses same bound as weights)
FC_Bias = kaiming_uniform((2, 1), fan_in=6)

# Output all layers
np.set_printoptions(precision=4, suppress=True)
print("DW1 (2x5):\n", DW1)
print("\nPW1 (4x2):\n", PW1)
print("\nDW2 (4x3):\n", DW2)
print("\nPW2 (6x4):\n", PW2)
print("\nFC (2x6):\n", FC)
print("\nFC_Bias (2x1):\n", FC_Bias)
```

```
DW1 (2x5):
 [[-0.1122  0.4031  0.2075  0.0882 -0.3077]
 [-0.3077 -0.3953  0.3275  0.0904  0.1861]]

PW1 (4x2):
 [[-0.678   0.6646]
 [ 0.4701 -0.4068]
 [-0.45   -0.4477]
 [-0.2768  0.035 ]]

DW2 (4x3):
 [[-0.0786 -0.2411  0.1292]
 [-0.4163 -0.24   -0.1543]
 [-0.0507  0.3293 -0.3468]
 [ 0.0164  0.1067 -0.5237]]

PW2 (6x4):
 [[ 0.1075 -0.3295 -0.4349  0.4489]
 [ 0.4656  0.3084 -0.1954 -0.4023]
 [ 0.1842 -0.0598 -0.378  -0.0048]
 [-0.4656  0.4093 -0.2412  0.1625]
 [-0.1883  0.0201  0.0467 -0.3151]
 [ 0.4696  0.2751  0.4395  0.3948]]

FC (2x6):
 [[ 0.0799  0.3445 -0.336  -0.2482 -0.3713 -0.1426]
 [-0.0909 -0.1867  0.2684 -0.117  -0.1789  0.0349]]

FC_Bias (2x1):
 [[-0.2932]
 [ 0.2467]]
```

1.0.2 Step 2

Typically we would quantize our model, layer by layer. Since we have no training data or even context for this model, we will simply quantize the weights all at once. In a real model, this would have a significantly deleterious effect on performance, but is sufficient for our illustration.

[2]:
```
weights = {
    "DW1": DW1,
    "PW1": PW1,
    "DW2": DW2,
    "PW2": PW2,
    "FC": FC,
    "FC_Bias": FC_Bias
}

scales = {}
quantized_weights = {}

for name, W in weights.items():
    scale = 7 / np.max(np.abs(W))
    QW = np.floor(scale * W).astype(np.int8)
    scales[name] = scale
    quantized_weights[name] = QW

for name in weights:
    print(f"{name} scale factor: {scales[name]:.4f}")
    print(f"{name} quantized:\n{quantized_weights[name]}\n")
```

```
DW1 scale factor: 17.3641
DW1 quantized:
[[-2  7  3  1 -6]
 [-6 -7  5  1  3]]

PW1 scale factor: 10.3245
PW1 quantized:
[[-7  6]
 [ 4 -5]
 [-5 -5]
 [-3  0]]

DW2 scale factor: 13.3661
DW2 quantized:
[[-2 -4  1]
 [-6 -4 -3]
 [-1  4 -5]
 [ 0  1 -7]]

PW2 scale factor: 14.9068
PW2 quantized:
```

```
[[ 1 -5 -7  6]
 [ 6  4 -3 -6]
 [ 2 -1 -6 -1]
 [-7  6 -4  2]
 [-3  0  0 -5]
 [ 7  4  6  5]]

FC scale factor: 18.8516
FC quantized:
[[ 1  6 -7 -5 -7 -3]
 [-2 -4  5 -3 -4  0]]

FC_Bias scale factor: 23.8758
FC_Bias quantized:
[[-7]
 [ 5]]
```

1.0.3 Step 3

We define an input tensor sampled from a uniform distribution in int4

[3]:
```
x = np.array([[-2, -5, 4, 6, 2, -1, 4, -1],
              [-2, 1, -6, -2, 2, 2, -1, -4]])
print(x)
```

```
[[-2 -5  4  6  2 -1  4 -1]
 [-2  1 -6 -2  2  2 -1 -4]]
```

1.0.4 Step 4

We pad the input tensor with zeros to prepare for our first depthwise convolution

[4]:
```
# Pad input along the second axis
kernel_size = 5
x_padded = np.pad(x, ((0, 0), (kernel_size // 2, kernel_size // 2)),␣
 ↪mode='constant')
print(x_padded)
```

```
[[ 0  0 -2 -5  4  6  2 -1  4 -1  0  0]
 [ 0  0 -2  1 -6 -2  2  2 -1 -4  0  0]]
```

1.0.5 Step 5

We perform the first depthwise convolution. Note the accumulation width.

[5]:
```
DW1_Q = quantized_weights["DW1"]
kernel_size = DW1_Q.shape[1]

# Output array (same shape as input)
```

```
y = np.zeros_like(x, dtype=np.int32)

# Apply depthwise convolution
for j in range(DW1_Q.shape[0]):  # number of channels
    kernel = DW1_Q[j]
    y[j] = [np.dot(x_padded[j, i:i+kernel_size], kernel) for i in range(x.
 ↪shape[1])]

print(y)
```

```
[[-35 -61 -25  64  15   9   0  27]
 [-27   7 -21  34  59  -5 -35 -25]]
```

1.0.6 Step 6

Apply arithmetic right shift minimally to ensure the maximum value in y ends up in int4. Note that we nearly end up in int3 because 64 is outside the bounds of int7 by 1. If we had maxed out at 63, we could have just shifted by 3.

[6]:
```
y_shift1 = y >> 4
print(y_shift1)
```

```
[[-3 -4 -2  4  0  0  0  1]
 [-2  0 -2  2  3 -1 -3 -2]]
```

1.0.7 Step 7

Perform a pointwise convolution. Once again, not the accumulation width.

[7]:
```
# Assume x is shape (C_in, W)
# Assume quantized_weights["PW1"] is shape (C_out, C_in)

PW1_Q = quantized_weights["PW1"]
C_out, C_in = PW1_Q.shape
W = y_shift1.shape[1]

# Output: shape (C_out, W)
y_pw1 = np.zeros((C_out, W), dtype=np.int32)

for i in range(W):  # For each spatial location
    for j in range(C_out):  # For each output channel
        y_pw1[j, i] = np.dot(PW1_Q[j], y_shift1[:, i])

print(y_pw1)
```

```
[[  9  28   2 -16  18  -6 -18 -19]
 [ -2 -16   2   6 -15   5  15  14]
 [ 25  20  20 -30 -15   5  15   5]
 [  9  12   6 -12   0   0   0  -3]]
```

1.0.8 Step 8

Shift to regain int4 range.

[8]:
```
y_shift2 = y_pw1 >> 2
print(y_shift2)
```

```
[[ 2  7  0 -4  4 -2 -5 -5]
 [-1 -4  0  1 -4  1  3  3]
 [ 6  5  5 -8 -4  1  3  1]
 [ 2  3  1 -3  0  0  0 -1]]
```

1.0.9 Step 9

ReLU

[9]:
```
relu1=np.maximum(0,y_shift2)
print(relu1)
```

```
[[2 7 0 0 4 0 0 0]
 [0 0 0 1 0 1 3 3]
 [6 5 5 0 0 1 3 1]
 [2 3 1 0 0 0 0 0]]
```

1.0.10 Interlude: MaxPool Method

We define a method to perform max pooling for ease of use later.

[10]:
```
def maxpool1d(x, kernel_size=2, stride=2):
    C, W = x.shape
    assert W % stride == 0, "Width must be divisible by stride for this simple␣
 ↪implementation."
    out_width = W // stride

    pooled = np.zeros((C, out_width), dtype=x.dtype)
    for i in range(out_width):
        start = i * stride
        end = start + kernel_size
        pooled[:, i] = np.max(x[:, start:end], axis=1)
    return pooled
```

1.0.11 Step 10

MaxPool, kernel = 2, stride = 2

[11]:
```
pool1 = maxpool1d(relu1)
print(pool1)
```

```
[[7 0 4 0]
 [0 1 1 3]
```

```
 [6 5 1 3]
 [3 1 0 0]]
```

1.0.12 Step 11

Pad our intermediate values before the next depthwise convolution.

[12]:
```
# Pad input along the second axis
DW2_Q = quantized_weights["DW2"]
kernel_size = DW2_Q.shape[1]
pool1_padded = np.pad(pool1, ((0, 0), (kernel_size // 2, kernel_size // 2)),␣
 ↪mode='constant')
print(pool1_padded)
```

```
[[0 7 0 4 0 0]
 [0 0 1 1 3 0]
 [0 6 5 1 3 0]
 [0 3 1 0 0 0]]
```

1.0.13 Step 12

We perform the second depthwise convolution

[13]:
```
# Output array (same shape as input)
dw2 = np.zeros_like(pool1, dtype=np.int32)

# Apply depthwise convolution
for j in range(DW2_Q.shape[0]):  # number of channels
    kernel = DW2_Q[j]
    dw2[j] = [np.dot(pool1_padded[j, i:i+kernel_size], kernel) for i in␣
 ↪range(pool1.shape[1])]

print(dw2)
```

```
[[-28 -10 -16  -8]
 [ -3  -7 -19 -18]
 [ -1   9 -16  11]
 [ -4   1   0   0]]
```

1.0.14 Step 13

Once again, we right shift to regain int4 representation.

[14]:
```
y_shift3 = dw2 >> 2
print(y_shift3)
```

```
[[-7 -3 -4 -2]
 [-1 -2 -5 -5]
 [-1  2 -4  2]
 [-1  0  0  0]]
```

1.0.15 Step 14

We perform our second pointwise convolution

[15]:
```
PW2_Q = quantized_weights["PW2"]
C_out, C_in = PW2_Q.shape
W = y_shift3.shape[1]

# Output: shape (C_out, W)
y_pw2 = np.zeros((C_out, W), dtype=np.int32)

for i in range(W):  # For each spatial location
    for j in range(C_out):  # For each output channel
        y_pw2[j, i] = np.dot(PW2_Q[j], y_shift3[:, i])

print(y_pw2)
```

```
[[ -1  -7  49   9]
 [-37 -32 -32 -38]
 [ -6 -16  21 -11]
 [ 45   1  14 -24]
 [ 26   9  12   6]
 [-64 -17 -72 -22]]
```

1.0.16 Step 15

Right shift back into int4

[16]:
```
y_shift3 = y_pw2 >> 4
print(y_shift3)
```

```
[[-1 -1  3  0]
 [-3 -2 -2 -3]
 [-1 -1  1 -1]
 [ 2  0  0 -2]
 [ 1  0  0  0]
 [-4 -2 -5 -2]]
```

1.0.17 Step 16

ReLU activation a second time.

[17]:
```
relu2=np.maximum(0,y_shift3)
print(relu2)
```

```
[[0 0 3 0]
 [0 0 0 0]
 [0 0 1 0]
 [2 0 0 0]
 [1 0 0 0]
 [0 0 0 0]]
```

1.0.18 Step 17

Now, our second MaxPool. Same kernel and stride as before.

[18]:
```
pool2 = maxpool1d(relu2)
print(pool2)
```

```
[[0 3]
 [0 0]
 [0 1]
 [2 0]
 [1 0]
 [0 0]]
```

1.0.19 Step 18

We now global average pool. This is the sum of each row, right shifted by 1.

[19]:
```
gap = np.array([[1],[0],[0],[1],[0],[0]])
print(gap)
```

```
[[1]
 [0]
 [0]
 [1]
 [0]
 [0]]
```

1.0.20 Step 19

Apply the fully connected layer to arrive at our final logits.

[20]:
```
fc_output = np.dot(quantized_weights["FC"], gap) + quantized_weights["FC_Bias"]

print(fc_output)
```

```
[[-11]
 [  0]]
```

1.0.21 Step 20

ArgMax. Identify the largest logit as our final inference

[21]:
```
output = 0
print(output)
```

```
0
```